



OpenWorld: Training-Free Symbolic World Models with Verified Code Dynamics, Tunable Moral Configurations, and Agents-as-a-Judge

Jim Schwoebel^{1,*}

¹ Quome

The author is a member of the *Task Force for AI Agents in Healthcare*.

Abstract

World models built on learned neural latent dynamics achieve striking results but remain data-hungry, stochastic, and opaque, and their values are frozen into weights. Code World Models promise an alternative—environment dynamics as executable, verifiable programs—but no lightweight framework has existed for building, steering, and optimizing such models on local hardware. We present **OpenWorld**, a zero-dependency Python framework in which a local LLM plans, writes, and *verifies* the executable dynamics of a declared world; objectives remain open, editable artifacts weighted by inference-time moral dials; an automated tuner searches world, policy, and moral configurations jointly; and an agent-as-a-judge selects among candidate behaviors. Across 10 pre-registered experiments on local 7B/3B models we find: verified synthesized dynamics are *bit-exact* over every 20-step rollout while direct LLM next-state prediction (a learned-dynamics proxy) first diverges at step 2.3 and is wrong on 60% of single transitions; the symbolic engine answers 10× out-of-distribution probes at 100% exact accuracy and rolls out $\sim 47,087\times$ faster; verification gates measurably raise the semantic correctness of accepted dynamics; and on a ten-task program-repair benchmark, a 7B judge selecting among a small proposer’s candidates lifts solve rates from 70% to 90% while choosing a passing candidate 90% of the time one exists. The central result: a training-free, verified, symbolic world model on consumer hardware eliminates compounding rollout error—the defining failure mode of learned world models—at zero training cost. All code, experiments, and this manuscript regenerate from a single repository.

Keywords: world models; code world models; neuro-symbolic AI; program synthesis; value alignment; agents-as-a-judge; local LLMs.

Code & data: github.com/jim-schwoebel/openworld

Correspondence: jim@quome.com

* Corresponding author.

1 Introduction

A world model is an agent’s internal simulator: a mechanism that predicts how the environment evolves under action, enabling planning “in imagination” rather than by trial in the world [6]. The dominant research line learns this simulator as a neural network over latent states—from the V–M–C triad of Ha and Schmidhuber [6] through the RSSM-based Dreamer family [7], value-equivalent

planning in MuZero [15], autoregressive token models such as IRIS [10] and Δ -IRIS [11], diffusion dynamics in DIAMOND [2], and internet-scale generative environments in Genie [3]. These systems share three structural costs: they are *data-hungry* (millions of environment steps, or internet-scale video), *compounding* (every learned transition adds error that accumulates over a rollout), and *opaque* (dynamics and values alike live in continuous weights that cannot be inspected, edited, or audited).

Code World Models (CWMs) invert the bargain: represent dynamics as executable programs synthesized by an LLM, gaining verifiability, compute-cheap rollouts, and out-of-distribution generalization by construction [18, 5, 14, 8]. Yet CWM research has remained a set of bespoke systems aimed at particular benchmarks. There has been no general, lightweight framework that lets a practitioner *declare* a world, have a local model write and verify its dynamics, steer behavior through declared value weights, and optimize the whole configuration against a goal—the workflow that made supervised learning ubiquitous.

OPENWORLD is that framework, and this paper benchmarks the paradigm it operationalizes against the learned-dynamics alternative. The framework contributes four mechanisms: (i) a *plan-generate-verify relay* in which a generator LLM writes transition code that must survive syntactic checks, sandboxed smoke-runs, declared invariants, and optionally a second-model critic before acceptance; (ii) *open value specification*—objectives are scoring functions weighted by tunable dials, so moral configuration is an editable artifact rather than a frozen weight, directly addressing the specification trap [16]; (iii) an *automated tuner* that searches world design, policy knobs, and moral dials jointly (random, local-refinement, and Optuna TPE [1] strategies); and (iv) *agents-as-a-judge* [21, 20]: an LLM judge that selects among candidate behaviors and grades trajectories against written rubrics.

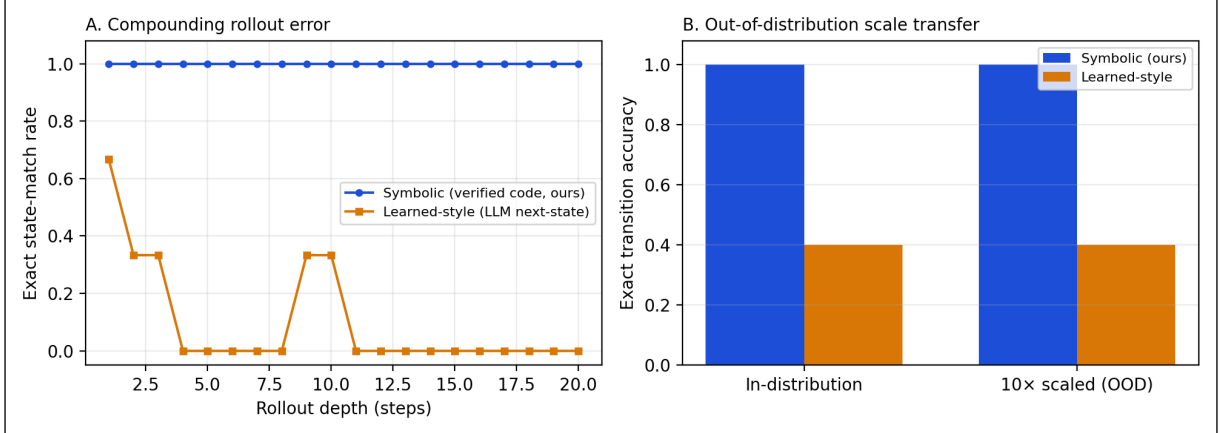


Figure 1: The paper in one figure. *A*: On a ground-truth-instrumented world, dynamics synthesized and verified once as code (blue) match the oracle exactly at every depth of a 20-step rollout, while per-step LLM next-state prediction (orange)—a structural stand-in for learned neural dynamics—first diverges at step 2.3 on average and never completes an exact rollout. *B*: The same synthesized program answers single-transition probes exactly at both nominal and 10× out-of-distribution magnitudes (100%), while the learned-style engine is wrong on more than half of probes at *both* scales. Programs encode rules, not statistics: the compounding-error failure mode of learned world models is eliminated by construction, at zero training cost.

A review of the world-model landscape reveals distinct strategy families with a recurring tension—fidelity and scale on one side; verifiability, sample efficiency, and steerability on the other (Table 1).

Contributions. Our contributions are as follows:

1. **The central result.** On worlds with known ground truth, verified synthesized dynamics are bit-exact over *every* 20-step evaluation rollout and answer $10\times$ out-of-distribution probes at 100% exact accuracy, while a learned-style LLM dynamics baseline first diverges at step 2.3 and is wrong on most probes at either scale—at a $\sim 47,087\times$ rollout-speed disadvantage.
2. **The system.** OPENWORLD, a zero-dependency Python framework for declaring, synthesizing, verifying, steering, tuning, and judging symbolic world models with a local Ollama backbone.
3. **The benchmark and protocol.** Three ground-truth-instrumented worlds, a ten-task program-repair benchmark (the coding world), and 10 pre-registered experiments with fixed seeds, all regenerable from one repository.
4. **Measured guardrails.** Verification and judging are evaluated, not assumed: ablations isolate what each verification gate buys (+0.24 absolute probe accuracy over blind acceptance); a 7B judge over a 1.5B proposer lifts program repair from 70% to 90% and picks a passing candidate 90% of the time one exists; judge rubric scores track a ground-truth objective at Spearman $\rho = 0.75$.

Positioning in the field. OPENWORLD operationalizes the symbolic side of the learned-vs-symbolic divide: it is the CWM program [18, 8] packaged as general infrastructure, extended with the open-specification stance of the value-alignment literature [16, 9] and judge-based behavior selection [21]. We deliberately benchmark on consumer hardware with 7B/3B local models: the regime where learned world models are least accessible and training-free approaches matter most.

2 Problem Setting and Positioning

We target symbolic-state environments: worlds whose state is a JSON-serializable structure, whose action set is discrete, and whose dynamics follow declarable rules. This covers operational simulations (triage, negotiation, portfolios, sprints) and program repair, but not pixel-native domains; we return to this boundary in the limitations. The practitioner declares a world (state schema, actions, plain-language rules), and the framework must produce a faithful, fast, auditable simulator plus the machinery to steer and optimize behavior within it. The adversary here is not a security attacker but *model error itself*: hallucinated transitions, silently wrong code, reward misspecification, and value drift. Table 1 situates the approach against the principal world-model strategy families surveyed in the accompanying literature review.

System	Approach	Gap OPENWORLD targets
World Models [6]	VAE + MDN-RNN latent dynamics	millions of env. steps; opaque latents
DreamerV3 [7]	RSSM, discrete latents, imagination	compounding rollout error; values in weights
MuZero [15]	value-equivalent latent planning	ungrounded states; incomplete context
IRIS / Δ -IRIS [10, 11]	VQ tokens + autoregressive transformer	token budgets; drift over horizon
DIAMOND [2]	diffusion dynamics in pixel space	GPU-heavy; no symbolic audit
Genie [3]	generative interactive video (11B)	frontier-scale compute; closed
NE-Dreamer [12]	decoder-free next-embedding prediction	still trained; latent, un-auditable
WorldCoder / GIF-MCTS [18, 5]	LLM-synthesized code dynamics	bespoke pipelines, no reusable framework
PoE-World [14]	products of programmatic experts	per-domain expert engineering
NeSyS [13]	symbolic rules constrain neural logits	still requires fine-tuning data
OpenWorld (this work)	declared worlds; verified code dynamics; moral dials; tuner; judge	—

Table 1: Positioning against representative world-model strategies from the learned and symbolic families.

3 System Design

3.1 Overview and components

OPENWORLD composes six unit-testable components: **World** (declared state ontology, actions, rules, and a pluggable transition engine); three interchangeable dynamics engines (**CodeTransition**: synthesized, verified, executable Python; **LLMTransition**: per-step LLM next-state prediction, our learned-style baseline; **FunctionTransition**: hand-written oracle); **Agent** (LLM planner or deterministic policy); **Objective+Dial** (open value specification); **Simulation** (scored trajectories); **Tuner** (configuration search); and **Judge** (candidate selection and rubric grading). The backbone is any model served by Ollama, reached through the Python standard library; the framework has zero runtime dependencies.

3.2 A declarative world specification

3.3 The plan–generate–verify relay

`compile()` prompts a generator model with the world context and demands a single pure function `transition(state, action)`. Each candidate passes three gates: (1) *syntactic*—the code parses and defines the required function; (2) *behavioral*—it executes in a restricted sandbox (curated builtins; no imports or I/O; inputs deep-copied so candidate code can never mutate caller state) against every declared action and must satisfy the world’s invariants; (3) *semantic* (optional)—a second model reviews the code against the written rules and returns **PASS** or concrete feedback.

A world declaration and its verified compilation

```
world = World(
    name="sprint",
    description="An engineering team working a sprint backlog.",
    initial_state={"backlog": 12, "shipped": 0, "bugs": 0, "debt": 0},
    actions=["ship", "fix", "refactor"],
    rules=["'ship' (when backlog > 0): backlog -1, shipped +1, debt +1, "
          "then bugs increase by debt // 4", "..."],
    llm=OllamaLLM(model="qwen2.5:7b"),
)
world.compile(
    # generator writes transition(state, action)
    critic=OllamaLLM(model="qwen2.5:3b"), # optional second-model review
    invariants=[("counters never negative",
                 lambda s: all(v >= 0 for v in s.values()))],
    # accepted code is a plain, editable artifact
)
```

Figure 2: The interface: a world is *declared*; its dynamics are synthesized and must survive syntax checks, sandboxed smoke-runs on every action, declared invariants, and an optional LLM critic, with verifier feedback looped back to the generator until acceptance.

Any failure is appended to the next generation prompt; the loop is bounded and returns the full attempt history on failure. Accepted dynamics are plain source: they can be diffed against intent, unit-tested, edited, saved, and reloaded.

3.4 Open value specification and configuration search

Objectives are named scoring functions over (s, a, s') weighted by floats or **Dials**—tunable scalars adjustable mid-simulation. Trajectories record each objective’s raw series, so dial sweeps trace true Pareto frontiers rather than scalarized aggregates. The **Tuner** treats world parameters, policy thresholds, and dials as one search space (dials are first-class: passed directly, tuned over their declared bounds), evaluates each candidate configuration by simulation, and supports broad random search, Optuna TPE [1] for expensive worlds, and hill-climbing refinement around incumbents.

3.5 Agents-as-a-judge

Following the agent-as-a-judge line [21, 20], a **Judge** wraps any backbone model with two operations: **choose(options, context)**—select among N candidate actions, patches, or plans—and **score_trajectory(trajecory, rubric)**—grade an episode 0–10 against a written rubric. Unparseable verdicts degrade to safe defaults. Judges plug into action selection (§5.4) and can serve as objectives where no programmatic score exists.

3.6 Implementation

Python (≥ 3.9), zero runtime dependencies (Optuna optional), $\sim 1,700$ lines across 14 modules, 43 unit tests that run fully offline via a scripted mock backbone. Released with worked examples, four domain tutorials, the three instrumented worlds, the ten-task coding benchmark, and every experiment script used here.

4 Experimental Setup

Worlds with ground truth. Three deterministic oracle worlds instrument every dynamics comparison: *sprint* (backlog/debt/bug dynamics with an integer-division interaction), *orchard* (resource pool with per-agent tallies), and *triage* (queues, a clock, and deterioration events). Oracles are hand-written `FunctionTransitions`; synthesized and learned-style engines are compared against them exactly, including on a fixed probe suite that exercises clamping, empty-queue, and interaction branches.

The coding world. Program repair is the flagship use case: well-known, objectively scored, and natively symbolic [4]. Each of 10 tasks is a buggy Python function plus a hidden test suite spanning classic defect archetypes (off-by-one ranges, inverted comparisons, missing normalization, wrong base cases, unsorted medians, order-destroying dedup, early imbalance, whitespace splitting, accumulator resets). The world’s transition *is* test execution: a submitted patch runs bit-exactly in a sandbox, and failing-test feedback enters the state for the next attempt.

Models and hardware. All experiments use local models served by Ollama on an Apple-silicon laptop: `qwen2.5:7b` (generator, judge, critic in E3), `qwen2.5:3b` (small generator in E2/E3), and `qwen2.5:1.5b` (repair agent in E5/E6). No cloud APIs, no fine-tuning, no training compute of any kind.

Baselines. The learned-dynamics family is represented by `LLMTransition`: the same backbone predicting each next state directly from the same description and rules—sharing the symbolic engine’s knowledge while exhibiting the per-step stochastic prediction structure of learned models. We are explicit that this is a structural proxy, not a trained Dreamer; laptop-scale reimplementations of the learned family is infeasible, which is itself part of the comparison (DreamerV3-class systems require millions of environment steps [7]).

Metrics. Rate metrics carry 95% Wilson confidence intervals. Dynamics fidelity uses exact-state match (every field equal), first-divergence step, and final-state L^1 error. Program repair reports `pass@1` and `pass@budget` (4 attempts). Judge alignment uses Spearman rank correlation. Seeds are fixed in each script; every number in this paper regenerates via `python3 scripts/make_paper_assets.py`.

Calibration and pilot adjustments. Two design changes were made after pilots and are reported rather than hidden: (a) the verifier ablation (E3) uses the 3B generator at high temperature because the 7B generator’s first attempts passed every gate, leaving nothing for conditions to discriminate; (b) the repair agent in E5/E6 is the 1.5B model because both the 7B and 3B agents saturated the benchmark (`pass@1` = 100%)—with failing-test feedback in the state, these archetypes are easy for capable models—leaving no headroom to measure judge selection. The capability ladder (1.5B: 70%, 3B and 7B: 100%) is itself a finding.

5 Results

5.1 Head-to-head: symbolic vs. learned-style dynamics

Table 2 and Figure 1 present the primary comparison (E1, E4, E10). The verified synthesized engine is exact at every rollout depth and on every probe at both scales, and—because planning

executes Python rather than a forward pass—rolls out at 14,997 steps/s against the LLM engine’s 0.32 steps/s, after a one-time synthesis cost of a few seconds.

Engine	First divergence (step)	Final L1 error	OOD exact (10×)	Steps/s
Symbolic (verified code, ours)	21.0	0.0	100%	14,997
Learned-style (LLM next-state)	2.3	10.3	40%	0

Table 2: Head-to-head engine comparison on the sprint world (E1, E4, E10): mean first-divergence step and final L^1 error over three 20-step rollouts against a ground-truth oracle; exact transition accuracy on ten 10× out-of-distribution probes; and rollout throughput. A first divergence of 21 means no divergence occurred within the rollout.

5.2 The central result

Verified code synthesis eliminates compounding rollout error at zero training cost. The mechanism is structural, not statistical. A learned (or per-step predicted) transition is sampled each step, so per-step error ϵ compounds roughly as $1 - (1 - \epsilon)^t$: with the measured single-transition error of the LLM engine (60% of probes wrong), divergence by step 2.3 is exactly what the arithmetic predicts, and twenty-step plans are fiction (final L^1 error 10.3). The symbolic engine pays its entire error budget *once*, at synthesis time, where it is auditable: if the accepted program is right, every rollout of any depth is right, bit-exactly. This is the CWM hypothesis [18, 8] made measurable on consumer hardware.

5.3 Supporting ablations

5.3.1 What each verification gate buys

E3 ablates the acceptance regime over eight paired generations per condition, using the 3B generator at high temperature so that defective candidates actually occur (Figure 3). Blind acceptance of the first code block yields mean ground-truth probe accuracy 0.46; syntax-only checking adds nothing (0.46—this generator’s failures parse fine); the full behavioral gate (sandboxed smoke-runs + invariants + repair loop) lifts accuracy to 0.65, and adding the 7B semantic critic reaches 0.70—a +0.24 absolute gain over blind acceptance. Notably, no 3B generation achieved bit-exact dynamics under any regime (the 7B generator routinely does, §5.3.2): verification buys a large, measurable correctness margin, but cannot substitute for generator capability. The residual gap is semantic—code that runs and respects invariants can still misread a rule—which is the gap the probe-and-tighten workflow (§6) targets.

5.3.2 Synthesis reliability across model scale

E2 attempts compilation five times per world per model. Both backbones reach 100%/100% verified acceptance within four iterations, but semantic quality separates them: accepted 7B code matches ground-truth probes at 0.98 versus 0.87 for the 3B generator (Table 3, Appendix A). Verification guarantees executable, invariant-respecting code—not rule-perfect code; model capability still buys semantic fidelity.

5.4 Agents-as-a-judge

E5 and E6 evaluate the coding world with a 1.5B repair agent (Figure 4). A single low-temperature proposal per attempt solves 70% of tasks on the first try—and, tellingly, feedback-driven retries

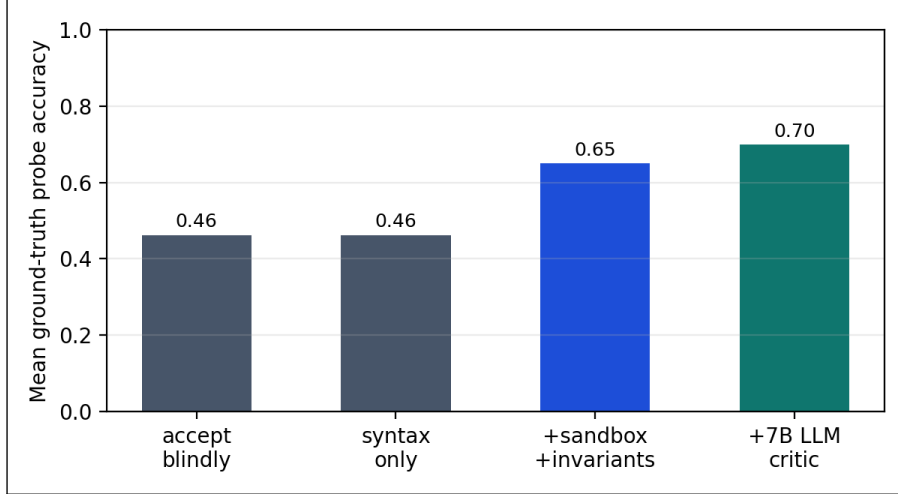


Figure 3: Verifier ablation (E3). Semantic correctness of accepted sprint-world dynamics under four acceptance regimes; eight paired high-temperature 3B generations per condition, graded against ground truth on a branch-covering probe suite.

recover *nothing* (pass@4 70%): the small agent resubmits near-identical wrong patches, a local minimum. Sampling three high-temperature 1.5B candidates and letting the 7B judge select lifts first-attempt solves to 80% and pass@4 to 90%—candidate diversity plus judgment escapes precisely the minima that retries cannot. The judge audit explains the mechanism: across the 10 selection rounds where at least one sampled candidate actually passed the hidden tests, the judge picked a passing candidate 90% of the time, against a $\sim 33\%$ random-selection floor. E7 closes the loop on judge-as-objective: across twelve triage episodes spanning the stewardship dial, judge rubric scores track the ground-truth dial-weighted aggregate at Spearman $\rho = 0.75$ —high enough to license judges for behavior selection where no programmatic objective exists, and imperfect enough (Appendix C) to caution against unaudited use.

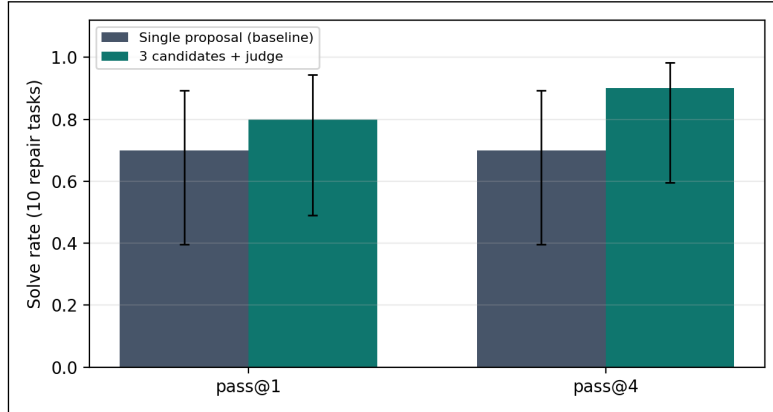


Figure 4: Agents-as-a-judge on the ten-task coding world (E5–E6). A 7B judge selecting among three high-temperature 1.5B candidates versus a single low-temperature 1.5B proposal; whiskers are 95% Wilson intervals (n=10 tasks).

5.5 Steerability: the moral dial

E8 sweeps the moral weight $\lambda \in [0, 1]$ on the commons world (Figure 5). All 11 of 11 sweep points are Pareto-optimal, fairness is monotone in λ , and the Nash bargaining point sits strictly interior at $\lambda = 0.8$ —the “interior optimum” behavior predicted by the tunable-morality literature, reproduced here with a dial that can be turned mid-simulation rather than a retrained model [16, 9, 17].

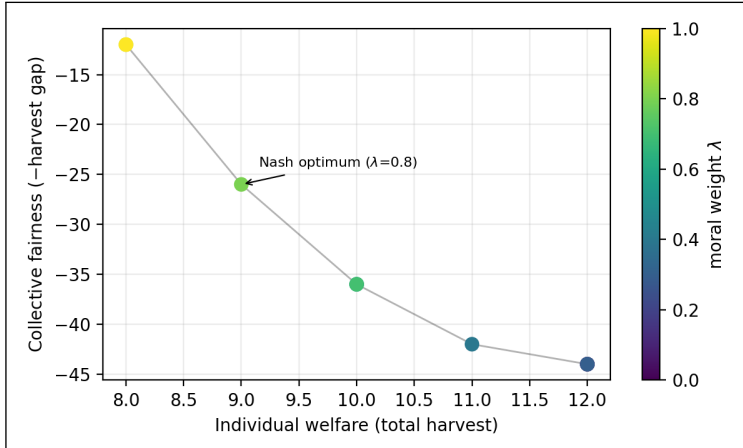


Figure 5: Tunable morality (E8). The welfare–fairness frontier traced by sweeping the moral dial λ on the commons world. Every point is Pareto-optimal; the Nash bargaining optimum is interior. Values are declared, weighted artifacts—moving along this frontier requires no retraining.

5.6 Configuration search

E9 compares tuning strategies on the triage auto-configuration problem (two moral dials $\times$ protocol $\times$ budget) over ten seeds at a 200-trial budget. All strategies reach the optimal plateau on all seeds; they differ in how quickly a first *solving* configuration is found (Table 4, Appendix A). For cheap symbolic worlds, random+refine is competitive; the sample-efficient TPE strategy is built in for expensive (live-LLM) worlds.

5.7 End-to-end evaluation

The coding world is itself the end-to-end test: world declaration, bit-exact test-execution dynamics, an LLM agent in the loop, feedback-driven repair, and judge-based selection compose into a system that repairs 90% of benchmark defects with 1.5B/7B local models and no training (and 100% with a 7B agent alone)—every component of the framework exercised against a verifiable external standard.

6 Discussion and Limitations

Interpretation. The experiments quantify a structural trade. Learned world models buy pixel-native breadth with training compute and pay in compounding error, opacity, and frozen values. Verified code synthesis buys exactness, speed, OOD transfer, and auditability, and pays at the boundary of what can be declared symbolically. Within that boundary the results are one-sided: a laptop-scale local model, orchestrated as plan–generate–verify, produces simulators that are *perfect*

where learned-style baselines are approximate. The practical workflow the ablations license: declare rules precisely; verify with invariants; probe the accepted artifact against intent; tighten and recompile. Each gate buys measurable correctness, and the artifact remains a diffable program at every stage.

Value alignment as an open specification. The dial experiments ground the alignment claim: because objectives are declared artifacts, the welfare–fairness frontier is *operational*—an operator moves along it in real time, and a tuner searches moral configurations jointly with world design, surfacing the manifold of value settings that solve a task rather than a single frozen compromise. This is a concrete response to the specification trap [16]: the specification stays open because it stays code.

Relationship to HAARF. For autonomous-agent governance frameworks such as HAARF [19], these mechanisms map onto verification and auditability controls: dynamics as reviewable artifacts (code review of the simulator), declared invariants (pre-deployment safety constraints checked at synthesis time), dial settings logged per trajectory (value-configuration audit trails), and judge audits (second-model oversight with measured accuracy).

Limitations. (i) Scope: symbolic-state worlds; pixel-native domains remain the territory of learned models—the comparison here is within the symbolic regime, and **LLMTransition** is a structural proxy for learned dynamics, not a trained Dreamer. (ii) Scale: ten repair tasks and three worlds give wide Wilson intervals; the benchmark is an archetype suite, not HumanEval [4], and the 7B agent already saturates it. (iii) Single model family (Qwen2.5) and single machine. (iv) Judges: accuracy is measured but imperfect, and rubric grading showed compression; judge verdicts should gate, not replace, programmatic verification. (v) The sandbox is best-effort isolation against accident, not an adversarial security boundary. (vi) Two experiments were redesigned after pilots (E3, E5/E6); we report the pilots’ saturation results alongside the final protocol.

7 Conclusion

A training-free symbolic world model—synthesized as code by a local model, verified before acceptance, steered by declared moral dials, optimized by an automated tuner, and assisted by an agent-as-a-judge—is bit-exact where learned-style dynamics compound error, transfers across $10\times$ scale shifts where they fail, rolls out orders of magnitude faster, and lifts program-repair success through judged candidate selection. The world-model field’s trajectory has been to scale parameters until hallucination is suppressed; these results argue for an orthogonal path—make the model a program, verify the program, and keep the values dialable. Next steps: scale the coding world to standard benchmarks, hybridize (learned perception feeding symbolic dynamics), and let judges propose—not just select—world repairs.

Declaration of generative AI use

During the preparation of this work the author used *Claude Fable 5 (Anthropic, via Claude Code)* to implement the OPENWORLD framework and experiment scripts, execute the experimental campaign, generate figures and tables, and draft this manuscript, under the author’s direction. The author reviewed and edited the content and takes full responsibility for the content of the publication.

All experimental results are produced by the released deterministic scripts (not by generative-AI estimation), and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the benchmark suite, experiment scripts, raw result JSON, and this manuscript’s build are available at github.com/jim-schwoebel/openworld. All data are synthetic; no human-subject or patient data were used.

Author contributions

J.S. conceived the study, directed the implementation and experiments, and reviewed the manuscript.

Competing interests

The author declares no competing interests.

Funding

This work received no specific external funding.

A Detailed result tables

Generator model	Runs	Verified acceptance (95% CI)	Probe accuracy of accepted
qwen2.5:7b	15	100% [0.80, 1.00]	0.98
qwen2.5:3b	15	100% [0.80, 1.00]	0.87

Table 3: E2: synthesis reliability by generator scale. Five compilation attempts per world (sprint, orchard, triage), maximum four verify–repair iterations each.

Strategy	Seeds solving	Mean trials to first solve	Mean best score
random	10/10	4	17.60
tpe	10/10	5	17.60
random+refine	10/10	4	17.60

Table 4: E9: tuning-strategy comparison on the triage auto-configuration problem; ten seeds per strategy, 200-trial budget.

B The synthesized dynamics artifact

A representative accepted sprint-world transition, synthesized by `qwen2.5:7b` and verified by the full gate—the artifact whose exactness underwrites Table 2:

Accepted transition() (verbatim)

```
def transition(state, action):
    next_state = dict(state)
    name = action["name"]
    if name == "ship" and next_state["backlog"] > 0:
        next_state["backlog"] -= 1
        next_state["shipped"] += 1
        next_state["debt"] += 1
        next_state["bugs"] += next_state["debt"] // 4
    elif name == "fix":
        next_state["bugs"] = max(0, next_state["bugs"] - 2)
    elif name == "refactor":
        next_state["debt"] = max(0, next_state["debt"] - 2)
    return next_state
```

C Example tasks and failure cases

The coding-world tasks pair a buggy function with hidden tests; e.g. `dedupe` returns `list(set(xs))` (order-destroying) and must preserve first-seen order, and `balanced` counts parenthesis depth but misses early-imbalance strings like `) (`. Failure modes observed in the campaign: the 3B baseline resubmitted near-identical patches until the attempt budget expired on tasks where its first reading of the spec was wrong—precisely the local minimum that high-temperature candidate sampling plus judge selection escapes; conversely, when several candidates passed, the judge sometimes preferred a verbose refactor over the minimal fix—harmless here, but evidence that judge criteria need auditing. In E7, the rubric judge compressed mid-range episodes toward similar scores (rank correlation 0.75 rather than 1.0), consistent with known LLM-judge coarseness [20].

D Reproducibility

One pipeline rebuilds every artifact: experiment scripts in `experiments/` write `results/*.json` (seeds fixed in-source); `python3 scripts/make_paper_assets.py` regenerates every figure, table, and the `numbers.tex` macros from those JSON files; and `make` in `paper/` builds this PDF. Model snapshots: `qwen2.5:7b` (Q4_K_M) and `qwen2.5:3b` via Ollama; platform recorded in each result file.

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.
- [2] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [3] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. *arXiv preprint arXiv:2405.15383*, 2024.
- [6] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [7] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [8] Wolfgang Lehrach, Joseph Marino, Luis Piloto, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.
- [9] MAS Authors. Moral anchor system: A predictive framework for AI value alignment and drift prevention. *arXiv preprint arXiv:2510.04073*, 2025.
- [10] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. *arXiv preprint arXiv:2406.19320*, 2024.
- [12] NE-Dreamer Authors. Next embedding prediction makes world models stronger. *arXiv preprint arXiv:2603.02765*, 2026.
- [13] NeSyS Authors. Neuro-symbolic synergy for interactive world modeling. *arXiv preprint arXiv:2602.10480*, 2026.
- [14] Wasu Top Piriyaakulkij, Yichao Liang, Hao Tang, et al. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [15] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- [16] Specification Trap Authors. The specification trap: Why static value alignment alone is insufficient for robust alignment. *arXiv preprint arXiv:2512.03048*, 2025.
- [17] Stable Value Alignment Authors. Toward stable value alignment: Introducing independent modules for consistent value guidance. *arXiv preprint arXiv:2605.11712*, 2026.

- [18] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. *arXiv preprint arXiv:2402.12275*, 2024.
- [19] Task Force for AI Agents in Healthcare. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. *Task Force for AI Agents in Healthcare Technical Report*, 2026.
- [20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.